

Lecture 06 - Rasterization

This lecture explains how continuous primitives become discrete fragments on a grid. Rasterization is the bridge from geometric descriptions to sample-based image generation.

Source chunk	course_text_parts/03_lectures/06_rasterization.txt
Extracted slide pages	61
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Untitled slide
Source cue	The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.
Commentary	This slide is about Untitled slide. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Untitled slide' into a causal sentence instead of repeating the slide title?

Page 2	Rasterization 1/2
Source cue	- Pixels ... / - ... are picture elements A Pixel Is NotA Little Square, / A Pixel Is NotA Little Square, / - ... are the smallest elements of a raster image / A Pixel Is NotA Little Square!
Commentary	This slide is about Rasterization 1/2. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Pixels ... / - ... are picture elements A Pixel Is NotA Little Square, / A Pixel Is NotA Little Square, / - ... are the smallest elements of a raster image / A Pixel Is NotA Little Square!
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Rasterization 1/2'?

Page 3	Rasterization 2/2
Source cue	- Pixel-based displays require discretization of primitives into pixels / - Discretization is done by rasterization algorithms / - Rasterization is time-critical / - Avoid expensive operations / - Optimize for hardware implementation
Commentary	This slide is about Rasterization 2/2. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Pixel-based displays require discretization of primitives into pixels / - Discretization is done by rasterization algorithms / - Rasterization is time-critical / - Avoid expensive operations / - Optimize for hardware implementation
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Rasterization 2/2'?
Page 4	6.1 Line Rasterization
Source cue	6.2 Triangle Edge Rasterization / 6.3 Region Filling / 6.4 Scanline-Based Triangle Rasterization / 6.5 Block-Based Triangle Rasterization
Commentary	This slide is about 6.1 Line Rasterization. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: 6.2 Triangle Edge Rasterization / 6.3 Region Filling / 6.4 Scanline-Based Triangle Rasterization / 6.5 Block-Based Triangle Rasterization
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by '6.1 Line Rasterization'?
Page 5	6.1 Line Rasterization
Source cue	Drawing a one pixel thick line
Commentary	This slide is about 6.1 Line Rasterization. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: Drawing a one pixel thick line
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by '6.1 Line Rasterization'?
Page 6	Line Rasterization
Source cue	- Given two 2D endpoints in integer coordinates, / determine pixels depicting 1-pixel wide line segment between endpoints / - Trivial cases / - Horizontal line segments / - Vertical line segments

Commentary	This slide is about Line Rasterization. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: - Given two 2D endpoints in integer coordinates, / determine pixels depicting 1-pixel wide line segment between endpoints / - Trivial cases / - Horizontal line segments / - Vertical line segments
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Line Rasterization'?
Page 7	Quality Criteria
Source cue	- For all line segments with a slope $ m \leq 1$, / one pixel should be drawn per column / - For all line segments with a slope $ m \geq 1$, / one pixel per line should be drawn / $ m > 1$
Commentary	This slide is about Quality Criteria. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - For all line segments with a slope $ m \leq 1$, / one pixel should be drawn per column / - For all line segments with a slope $ m \geq 1$, / one pixel per line should be drawn / $ m > 1$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Quality Criteria'?
Page 8	Naïve Algorithm 1/2
Source cue	- Utilization of the Cartesian normal form of the straight line equation $y = m \cdot x + b$ / - Assumptions (without restriction of generality) / - Line segments lie between (x_0, y_0) and (x_1, y_1) / $0 \leq x_0 \leq x_1 \leq 1$
Commentary	This slide is about Naïve Algorithm 1/2. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Utilization of the Cartesian normal form of the straight line equation $y = m \cdot x + b$ / - Assumptions (without restriction of generality) / - Line segments lie between (x_0, y_0) and (x_1, y_1) / $0 \leq x_0 \leq x_1 \leq 1$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Naïve Algorithm 1/2'?
Page 9	Naïve Algorithm 2/2
Source cue	- Downsides / - Does not work for $x = x_0$ / $0 \leq x_0 \leq 1$ / - Does not work for $ m > 1$, since gaps arise / (solution: swap the roles of x and y for $ m > 1$)
Commentary	This slide is about Naïve Algorithm 2/2. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Downsides / - Does not work for $x = x_0$ / $0 \leq x_0 \leq 1$ / - Does not work for $ m > 1$, since gaps arise / (solution: swap the roles of x and y for $ m > 1$)

Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Naïve Algorithm 2/2' into a causal sentence instead of repeating the slide title?
Page 10	Digital Difference Analyzer (DDA) 1/3
Source cue	- Reuse the last computed y to avoid multiplications in the loop / $x \rightarrow x + 1 \Rightarrow (x, y) \rightarrow (x + 1, y + \blacksquare)$ / <code>void drawLine(int x0, int y0, int x1, int y1) { / float m = float(y1 - y0) / float(x1 - x0); // Calculate the slope / float y = y0; // Start y at y0</code>
Commentary	This slide is about Digital Difference Analyzer (DDA) 1/3. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Reuse the last computed y to avoid multiplications in the loop / $x \rightarrow x + 1 \Rightarrow (x, y) \rightarrow (x + 1, y + \blacksquare)$ / <code>void drawLine(int x0, int y0, int x1, int y1) { / float m = float(y1 - y0) / float(x1 - x0); // Calculate the slope / float y = y0; // Start y at y0</code>
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Digital Difference Analyzer (DDA) 1/3' into a causal sentence instead of repeating the slide title?
Page 11	Digital Difference Analyzer (DDA) 2/3
Source cue	- Algorithm can be extended to relax endpoint order / <code>void drawLine(int x0, int y0, int x1, int y1) { / int sx = (x0 < x1 ? 1 : -1); // Determine x direction / float sm = sx * float(y1 - y0) / float(x1 - x0); // Calculate slope multiplied by step / float y = y0; // Start y at y0</code>
Commentary	This slide is about Digital Difference Analyzer (DDA) 2/3. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Algorithm can be extended to relax endpoint order / <code>void drawLine(int x0, int y0, int x1, int y1) { / int sx = (x0 < x1 ? 1 : -1); // Determine x direction / float sm = sx * float(y1 - y0) / float(x1 - x0); // Calculate slope multiplied by step / float y = y0; // Start y at y0</code>
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Digital Difference Analyzer (DDA) 2/3' into a causal sentence instead of repeating the slide title?
Page 12	Digital Difference Analyzer (DDA) 3/3
Source cue	- Downsides / - Does not work for $x = x / 0$ 1 / - Does not work for $ \blacksquare > 1$, since gaps arise / (solution: swap the roles of x and y for $ \blacksquare > 1$)
Commentary	This slide is about Digital Difference Analyzer (DDA) 3/3. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Downsides / - Does not work for $x = x / 0$ 1 / - Does not work for $ \blacksquare > 1$, since gaps arise / (solution: swap the roles of x and y for $ \blacksquare > 1$)

Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Digital Difference Analyzer (DDA) 3/3' into a causal sentence instead of repeating the slide title?
Page 13	Midpoint-Line Algorithm 1/2
Source cue	- Assumption 1: $0 \leq x \leq 1$ / (without loss of generality, as otherwise mirroring on axes/diagonals) / - Lower left end point (x_0, y_0) / upper right end point (x_1, y_1)
Commentary	This slide is about Midpoint-Line Algorithm 1/2. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Assumption 1: $0 \leq x \leq 1$ / (without loss of generality, as otherwise mirroring on axes/diagonals) / - Lower left end point (x_0, y_0) / upper right end point (x_1, y_1)
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Midpoint-Line Algorithm 1/2'?
Page 14	Midpoint-Line Algorithm 2/2
Source cue	- Observation / - x is closer to the line if x is above the line / - y is closer to the line if y is below the line / - Vertical distance between pixel to chose / and the actual line segment is always < 0.5
Commentary	This slide is about Midpoint-Line Algorithm 2/2. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Observation / - x is closer to the line if x is above the line / - y is closer to the line if y is below the line / - Vertical distance between pixel to chose / and the actual line segment is always < 0.5
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Midpoint-Line Algorithm 2/2'?
Page 15	Implicit Function
Source cue	- Line equation usually given as explicit function $x = y = \frac{y - y_0}{y_1 - y_0} \cdot (x_1 - x_0) + x_0$ / $x = y = \frac{y - y_0}{y_1 - y_0} \cdot (x_1 - x_0) + x_0$, with $x = x - x_0$ and $y = y - y_0$ / - Line equation can also be written as implicit function $(x, y) / x, y = \frac{y - y_0}{y_1 - y_0} \cdot (x_1 - x_0) + x_0 = 0$
Commentary	This slide is about Implicit Function. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Line equation usually given as explicit function $x = y = \frac{y - y_0}{y_1 - y_0} \cdot (x_1 - x_0) + x_0$ / $x = y = \frac{y - y_0}{y_1 - y_0} \cdot (x_1 - x_0) + x_0$, with $x = x - x_0$ and $y = y - y_0$ / - Line equation can also be written as implicit function $(x, y) / x, y = \frac{y - y_0}{y_1 - y_0} \cdot (x_1 - x_0) + x_0 = 0$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.

Check yourself	Can you explain which samples/fragments are generated by 'Implicit Function'?
Page 16	Decision Variable d
Source cue	- Sign of d is sufficient to decide which point is chosen / - Define decision variable d as $d = d_1 = d_2(x + 1, y + 1) / d_1 - d_2$ / - If $d > 0$, select d_1 (below the line) / - If $d < 0$, select d_2 (above the line)
Commentary	This slide is about Decision Variable d . Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Sign of d is sufficient to decide which point is chosen / - Define decision variable d as $d = d_1 = d_2(x + 1, y + 1) / d_1 - d_2$ / - If $d > 0$, select d_1 (below the line) / - If $d < 0$, select d_2 (above the line)
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Decision Variable d ' into a causal sentence instead of repeating the slide title?
Page 17	Update of d for d_1
Source cue	- Previous decision variable was given by $d_1 = d_2(x + 1, y + 1) / d_1 - d_2$ / - Previous decision variable was given by $d_1 = d_2(x + 1, y + 1) / d_1 - d_2$
Commentary	This slide is about Update of d for d_1 . Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Previous decision variable was given by $d_1 = d_2(x + 1, y + 1) / d_1 - d_2$ / - Previous decision variable was given by $d_1 = d_2(x + 1, y + 1) / d_1 - d_2$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Update of d for d_1 ' into a causal sentence instead of repeating the slide title?
Page 18	Update of d for d_2
Source cue	- Previous decision variable was given by $d_1 = d_2(x + 1, y + 1) / d_1 - d_2$ / - Previous decision variable was given by $d_1 = d_2(x + 1, y + 1) / d_1 - d_2$
Commentary	This slide is about Update of d for d_2 . Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Previous decision variable was given by $d_1 = d_2(x + 1, y + 1) / d_1 - d_2$ / - Previous decision variable was given by $d_1 = d_2(x + 1, y + 1) / d_1 - d_2$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Update of d for d_2 ' into a causal sentence instead of repeating the slide title?
Page 19	Proceeding
Source cue	- In each step, the algorithm selects between two pixels d_1 and d_2 / based on sign of decision variable d / - Depending on the choice, decision variable is updated / by adding difference Δd_1 or Δd_2

Commentary	This slide is about Proceeding. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - In each step, the algorithm selects between two pixels ■■ and ■ / based on sign of decision variable ■ / - Depending on the choice, decision variable is updated / by adding difference $\Delta■■$ or $\Delta■$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Proceeding' into a causal sentence instead of repeating the slide title?
Page 20	Initializing ■
Source cue	- First midpoint ■ is located at $(x + 1, y +) / 0\ 0\ 1\ 1\ ■ / ■\ x + 1, y + = ■ \cdot x + 1 + ■ \cdot y + + ■ = ■ \cdot x + ■ \cdot y + ■ + ■ + / 0\ 0\ 0\ 0\ 0\ 0$
Commentary	This slide is about Initializing ■. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - First midpoint ■ is located at $(x + 1, y +) / 0\ 0\ 1\ 1\ ■ / ■\ x + 1, y + = ■ \cdot x + 1 + ■ \cdot y + + ■ = ■ \cdot x + ■ \cdot y + ■ + ■ + / 0\ 0\ 0\ 0\ 0\ 0$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Initializing ■' into a causal sentence instead of repeating the slide title?
Page 21	Using ■
Source cue	- Midpoint line algorithms uses ■ as follows / $■ = 2 \cdot ■y - ■x / ■■■■■■ / \Delta■■ = 2 \cdot ■y - ■x / \Delta■ = 2 \cdot ■y$
Commentary	This slide is about Using ■. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Midpoint line algorithms uses ■ as follows / $■ = 2 \cdot ■y - ■x / ■■■■■■ / \Delta■■ = 2 \cdot ■y - ■x / \Delta■ = 2 \cdot ■y$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Using ■'?
Page 22	Pseudo Code – Simple Version
Source cue	<pre>void midPointLine(int x0, int y0, int x1, int y1) { / int dx = x1 - x0; // Calculate delta x / int dy = y1 - y0; // Calculate delta y / int d = 2 * dy - dx; // Initial decision variable / int dE = 2 * dy; // Increment used for move to E</pre>
Commentary	This slide is about Pseudo Code – Simple Version. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: <pre>void midPointLine(int x0, int y0, int x1, int y1) { / int dx = x1 - x0; // Calculate delta x / int dy = y1 - y0; // Calculate delta y / int d = 2 * dy - dx; // Initial decision variable / int dE = 2 * dy; // Increment used for move to E</pre>

Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Pseudo Code – Simple Version' into a causal sentence instead of repeating the slide title?
Page 23	Example
Source cue	- Draw line from (5, 8) to (9, 11) / - Order of values for ■ / - 2, 0, 6, 4 / - Order of chosen pixels / - NE, E, NE, NE
Commentary	This slide is about Example. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Draw line from (5, 8) to (9, 11) / - Order of values for ■ / - 2, 0, 6, 4 / - Order of chosen pixels / - NE, E, NE, NE
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Example'?
Page 24	Evaluation
Source cue	- Necessary arithmetic operations / - As only integer values are used, only integer operations are necessary / - Additions/subtractions only and multiplication by 2 / (left shift by 1 bit position) / - No multiplications in the loop
Commentary	This slide is about Evaluation. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Necessary arithmetic operations / - As only integer values are used, only integer operations are necessary / - Additions/subtractions only and multiplication by 2 / (left shift by 1 bit position) / - No multiplications in the loop
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Evaluation' into a causal sentence instead of repeating the slide title?
Page 25	Pseudo Code - Reversible
Source cue	void drawLine(int x0, int y0, int x1, int y1) { / int dx = abs(x1 - x0); // Calculate absolute delta x / int sx = (x0 < x1 ? 1 : -1); // Set step x based on direction / int dy = abs(y1 - y0); // Calculate absolute delta y / int sy = (y0 < y1 ? 1 : -1); // Set step y based on direction
Commentary	This slide is about Pseudo Code - Reversible. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: void drawLine(int x0, int y0, int x1, int y1) { / int dx = abs(x1 - x0); // Calculate absolute delta x / int sx = (x0 < x1 ? 1 : -1); // Set step x based on direction / int dy = abs(y1 - y0); // Calculate absolute delta y / int sy = (y0 < y1 ? 1 : -1); // Set step y based on direction
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.

Check yourself	Can you turn 'Pseudo Code - Reversible' into a causal sentence instead of repeating the slide title?
Page 26	6.2 Triangle Edge Rasterization
Source cue	Drawing a one pixel thick triangle edge
Commentary	This slide is about 6.2 Triangle Edge Rasterization. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: Drawing a one pixel thick triangle edge
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by '6.2 Triangle Edge Rasterization'?
Page 27	Triangle Edges 1/2
Source cue	- Using midpoint line algorithm for triangle edge rasterization / - Provides pixels closest to triangle edge / - Does not consider if pixel lies inside or outside triangle / $\Rightarrow$ Edge pixels can penetrate into interior of adjacent triangles
Commentary	This slide is about Triangle Edges 1/2. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Using midpoint line algorithm for triangle edge rasterization / - Provides pixels closest to triangle edge / - Does not consider if pixel lies inside or outside triangle / $\Rightarrow$ Edge pixels can penetrate into interior of adjacent triangles
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Triangle Edges 1/2'?
Page 28	Triangle Edges 2/2
Source cue	- Midpoint line algorithm must consider inside vs. outside pixels / - Adaptation of the midpoint line algorithm / - Restriction to pixels strictly inside triangle / - Must be applied to line rasterization also for consistency reasons / - Definition of triangle inside:
Commentary	This slide is about Triangle Edges 2/2. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Midpoint line algorithm must consider inside vs. outside pixels / - Adaptation of the midpoint line algorithm / - Restriction to pixels strictly inside triangle / - Must be applied to line rasterization also for consistency reasons / - Definition of triangle inside:
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Triangle Edges 2/2'?
Page 29	Edge Rasterization 1/3

Source cue	- Triangle edge rasterization can exploit edge coherence / - Two edges of triangle intersect a scanline / - Same edges intersected by scanline y likely also intersect scanline $y + 1$ / - Due to coherence, intersections can be calculated incrementally / - For each increase of $y \rightarrow y + 1$
Commentary	This slide is about Edge Rasterization 1/3. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Triangle edge rasterization can exploit edge coherence / - Two edges of triangle intersect a scanline / - Same edges intersected by scanline y likely also intersect scanline $y + 1$ / - Due to coherence, intersections can be calculated incrementally / - For each increase of $y \rightarrow y + 1$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Edge Rasterization 1/3'?

Page 30	Edge Rasterization 2/3
Source cue	- Integer arithmetic can be used for (left) edges with slopes > 1 / - Consider $\Delta x / \Delta y = (x_{right} - x_{left}) / (y_{right} - y_{left})$ / - x is only changed by $\Delta x / \Delta y \Rightarrow x$ can be represented as a fraction with denominator $(y_{right} - y_{left})$ / - Let i be the integer part
Commentary	This slide is about Edge Rasterization 2/3. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Integer arithmetic can be used for (left) edges with slopes > 1 / - Consider $\Delta x / \Delta y = (x_{right} - x_{left}) / (y_{right} - y_{left})$ / - x is only changed by $\Delta x / \Delta y \Rightarrow x$ can be represented as a fraction with denominator $(y_{right} - y_{left})$ / - Let i be the integer part
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Edge Rasterization 2/3'?

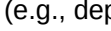
Page 31	Edge Rasterization 3/3
Source cue	<pre>void leftEdgeScan(int x0, int y0, int x1, int y1) { // slope > 1 / int d = y1 - y0; // denominator (Nenner) / int n = x1 - x0; // numerator (Zähler) / int a = x0; // integer part of x / int b = d; // fraction part of x plus d</pre>
Commentary	This slide is about Edge Rasterization 3/3. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: <pre>void leftEdgeScan(int x0, int y0, int x1, int y1) { // slope > 1 / int d = y1 - y0; // denominator (Nenner) / int n = x1 - x0; // numerator (Zähler) / int a = x0; // integer part of x / int b = d; // fraction part of x plus d</pre>
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Edge Rasterization 3/3'?

Page 32	6.3 Region Filling
Source cue	Homogeneous coloring of regions
Commentary	This slide is about 6.3 Region Filling. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: Homogeneous coloring of regions
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by '6.3 Region Filling'?
Page 33	Region Types 1/2
Source cue	- 4-connected / - All pixels of region can be reached by following movements / - Left, right, bottom, top / - 8-connected / - All pixels of region can be reached by following movements
Commentary	This slide is about Region Types 1/2. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - 4-connected / - All pixels of region can be reached by following movements / - Left, right, bottom, top / - 8-connected / - All pixels of region can be reached by following movements
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Region Types 1/2' into a causal sentence instead of repeating the slide title?
Page 34	Region Types 2/2
Source cue	- Examples / 4-connected 8-connected 8-connected
Commentary	This slide is about Region Types 2/2. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Examples / 4-connected 8-connected 8-connected
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Region Types 2/2' into a causal sentence instead of repeating the slide title?
Page 35	Region Spezifikation
Source cue	- Color-based / - Starting at pixel ■, determine largest contiguous region / which pixels have a specific color / - Color assignment done by flood fill algorithms / - Border-based
Commentary	This slide is about Region Spezifikation. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Color-based / - Starting at pixel ■, determine largest contiguous region / which pixels have a specific color / - Color assignment done by flood fill algorithms / - Border-based

Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Region Spezifikation' into a causal sentence instead of repeating the slide title?
Page 36	Flood Fill Algorithm
Source cue	<code>void floodFill4(int x, int y, Color oldC, Color newC) { / if ((x < 0) (x >= width)) return; // Return if x is out of bounds / if ((y < 0) (y >= height)) return; // Return if y is out of bounds / if (oldC == readPixel(x, y)) { // Check if the current color matches oldC / writePixel(x, y, newC); // Change the color of the pixel</code>
Commentary	This slide is about Flood Fill Algorithm. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: <code>void floodFill4(int x, int y, Color oldC, Color newC) { / if ((x < 0) (x >= width)) return; // Return if x is out of bounds / if ((y < 0) (y >= height)) return; // Return if y is out of bounds / if (oldC == readPixel(x, y)) { // Check if the current color matches oldC / writePixel(x, y, newC); // Change the color of the pixel</code>
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Flood Fill Algorithm' into a causal sentence instead of repeating the slide title?
Page 37	Boundary Fill Algorithm
Source cue	<code>void boundaryFill4(int x, int y, Color oldC, Color newC) { / if ((x < 0) (x >= width)) return; // Return if x is out of bounds / if ((y < 0) (y >= height)) return; // Return if y is out of bounds / Color c = readPixel(x, y); / if (c == oldC) return; // Return if pixel is a boundary pixel</code>
Commentary	This slide is about Boundary Fill Algorithm. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: <code>void boundaryFill4(int x, int y, Color oldC, Color newC) { / if ((x < 0) (x >= width)) return; // Return if x is out of bounds / if ((y < 0) (y >= height)) return; // Return if y is out of bounds / Color c = readPixel(x, y); / if (c == oldC) return; // Return if pixel is a boundary pixel</code>
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Boundary Fill Algorithm' into a causal sentence instead of repeating the slide title?
Page 38	Fill Algorithms
Source cue	- High number of recursions can lead to stack overflow / - Use smarter data structures, i.e., queues and stacks / Queue Stack
Commentary	This slide is about Fill Algorithms. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - High number of recursions can lead to stack overflow / - Use smarter data structures, i.e., queues and stacks / Queue Stack

Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Fill Algorithms' into a causal sentence instead of repeating the slide title?
Page 39	6.4 Scanline-Based Triangle Rasterization
Source cue	Segment-based rasterization of the simplest type of polygon
Commentary	This slide is about 6.4 Scanline-Based Triangle Rasterization. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: Segment-based rasterization of the simplest type of polygon
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by '6.4 Scanline-Based Triangle Rasterization'?
Page 40	Triangle Rasterization
Source cue	- Triangle rasterization / - Rasterize filled triangles is main task in real-time rendering / - Realization through graphics hardware desired / - General polygons split into triangles through tessellation / - Rasterization requires three steps
Commentary	This slide is about Triangle Rasterization. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Triangle rasterization / - Rasterize filled triangles is main task in real-time rendering / - Realization through graphics hardware desired / - General polygons split into triangles through tessellation / - Rasterization requires three steps
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Triangle Rasterization'?
Page 41	Scanline-Based Rasterization
Source cue	- Rasterization through scanline conversion exploits spatial coherence / - Pixels of a scanline are colored similarly / - Adjacent scanlines span similar pixels / - Adjacent scanlines are colored similarly / - Scanline segments can be calculated
Commentary	This slide is about Scanline-Based Rasterization. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Rasterization through scanline conversion exploits spatial coherence / - Pixels of a scanline are colored similarly / - Adjacent scanlines span similar pixels / - Adjacent scanlines are colored similarly / - Scanline segments can be calculated
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.

Check yourself	Can you explain which samples/fragments are generated by 'Scanline-Based Rasterization'?
Page 42	Algorithm Overview
Source cue	- Observation: triangle can be split into top an bottom half / - For each half two edge pixels and one segment exist / - Segments differ only slightly, depending on edge slope / - Procedure / - Sort triangle vertices based on y-coordinate
Commentary	This slide is about Algorithm Overview. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: - Observation: triangle can be split into top an bottom half / - For each half two edge pixels and one segment exist / - Segments differ only slightly, depending on edge slope / - Procedure / - Sort triangle vertices based on y-coordinate
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Algorithm Overview'?
Page 43	Slope Calculation
Source cue	- Calculate slope for each triangle edge: $1/\Delta = \Delta x / \Delta y$ / - $\Delta = (x - x_0) / (y - y_0)$ / $02\ 2\ 0\ 2\ 0$ / - $\Delta = (x - x_0) / (y - y_0)$ / $01\ 1\ 0\ 1\ 0$
Commentary	This slide is about Slope Calculation. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Calculate slope for each triangle edge: $1/\Delta = \Delta x / \Delta y$ / - $\Delta = (x - x_0) / (y - y_0)$ / $02\ 2\ 0\ 2\ 0$ / - $\Delta = (x - x_0) / (y - y_0)$ / $01\ 1\ 0\ 1\ 0$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Slope Calculation'?
Page 44	Pseudo Code
Source cue	<pre>void processTriangle(int x0, int y0, float delta02, float delta01, int Nbottom, int Ntop) { / float XLeft = x0; // Start position for left x / float XRight = x0; // Start position for right x / int Y = y0; // Start position for y / float DeltaLeft = delta02; // Delta for left x for bottom half</pre>
Commentary	This slide is about Pseudo Code. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: void processTriangle(int x0, int y0, float delta02, float delta01, int Nbottom, int Ntop) { / float XLeft = x0; // Start position for left x / float XRight = x0; // Start position for right x / int Y = y0; // Start position for y / float DeltaLeft = delta02; // Delta for left x for bottom half
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.

Check yourself	Can you explain which samples/fragments are generated by 'Pseudo Code'?
Page 45	Segment Rasterization
Source cue	- Each segment generates fragments / - Rendering pipeline processes generated fragments / (x , y) / (e.g., depth testing, alpha blending)  / (x , y)
Commentary	This slide is about Segment Rasterization. Read it as a data-flow explanation: scene or model data is processed step by step until valid framebuffer updates remain. The visible cue is: - Each segment generates fragments / - Rendering pipeline processes generated fragments / (x , y) / (e.g., depth testing, alpha blending)  / (x , y)
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Check yourself	Can you name the input and output representation for 'Segment Rasterization' in the rendering pipeline?
Page 46	Vertex Attribute Interpolation
Source cue	- During rasterization vertex attributes (e.g., color, depth) / must be interpolated over triangle / (x , y , z) /  / - Bilinear interpolation can be used
Commentary	This slide is about Vertex Attribute Interpolation. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - During rasterization vertex attributes (e.g., color, depth) / must be interpolated over triangle / (x , y , z) /  / - Bilinear interpolation can be used
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Vertex Attribute Interpolation'?
Page 47	Incremental Interpolation 1/3
Source cue	- Spatial coherence allows for an incremental attribute interpolation / - Example: consider triangle's plane equation for calculating z increments / - $\mathbf{a}x + \mathbf{b}y + \mathbf{c}z + \mathbf{d} = 0$ / - With plane coefficients $\mathbf{a}, \mathbf{b}, \mathbf{c}$: $\mathbf{a}, \mathbf{b}, \mathbf{c} = \mathbf{a} - \mathbf{a} \times (\mathbf{b} - \mathbf{b}) / 2 \ 0 \ 1 \ 2$
Commentary	This slide is about Incremental Interpolation 1/3. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Spatial coherence allows for an incremental attribute interpolation / - Example: consider triangle's plane equation for calculating z increments / - $\mathbf{a}x + \mathbf{b}y + \mathbf{c}z + \mathbf{d} = 0$ / - With plane coefficients $\mathbf{a}, \mathbf{b}, \mathbf{c}$: $\mathbf{a}, \mathbf{b}, \mathbf{c} = \mathbf{a} - \mathbf{a} \times (\mathbf{b} - \mathbf{b}) / 2 \ 0 \ 1 \ 2$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Incremental Interpolation 1/3'?

Page 48	Incremental Interpolation 2/3
Source cue	- Increments Δz_x and Δz_y are used to incrementally modify given z value, / i.e., when moving from (x, y) to $(x + 1, y)$ or $(x, y + 1)$ / - Case 1: next fragment in scanline (horizontal change by +1) / - $x \rightarrow x + 1 \Rightarrow z \leftarrow z + \Delta z_x$ / - $\Delta z_x / \Delta x + 1$ / -
Commentary	This slide is about Incremental Interpolation 2/3. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Increments Δz_x and Δz_y are used to incrementally modify given z value, / i.e., when moving from (x, y) to $(x + 1, y)$ or $(x, y + 1)$ / - Case 1: next fragment in scanline (horizontal change by +1) / - $x \rightarrow x + 1 \Rightarrow z \leftarrow z + \Delta z_x$ / - $\Delta z_x / \Delta x + 1$ / -
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Incremental Interpolation 2/3'?
Page 49	Incremental Interpolation 3/3
Source cue	- All vertex attributes can be interpolated incrementally analogous to depth / - Colors (r, g, b, a) / - Texture coordinates (u, v) / - ... / - For each attribute a define
Commentary	This slide is about Incremental Interpolation 3/3. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: - All vertex attributes can be interpolated incrementally analogous to depth / - Colors (r, g, b, a) / - Texture coordinates (u, v) / - ... / - For each attribute a define
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Incremental Interpolation 3/3'?
Page 50	6.5 Tile-Based Triangle Rasterization
Source cue	Parallel tile-based triangle rasterization
Commentary	This slide is about 6.5 Tile-Based Triangle Rasterization. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: Parallel tile-based triangle rasterization
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by '6.5 Tile-Based Triangle Rasterization'?
Page 51	Tile-Based Triangle Rasterization
Source cue	- Modern GPUs have a block-wise structure, / where one processor is responsible for one screen tile of adjacent pixels / - Segment-based triangled rasterization is difficult to parallelize / - Tile-based rasterization is a better fit with modern GPUs / - Advantage

Commentary	This slide is about Tile-Based Triangle Rasterization. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Modern GPUs have a block-wise structure, / where one processor is responsible for one screen tile of adjacent pixels / - Segment-based triangled rasterization is difficult to parallelize / - Tile-based rasterization is a better fit with modern GPUs / - Advantage
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Tile-Based Triangle Rasterization'?

Page 52	Edge-Based Triangle Representation 1/2
Source cue	- Edge-based area representation is possible through implicit edge functions / - Implicit edge function can be defined based on two edge vertices / $\blacksquare = (x, y)$ and $\blacksquare = (x, y)$ / 0 0 0 1 1 1 / - For a given coordinate (x, y) , the implicit edge function is defined as
Commentary	This slide is about Edge-Based Triangle Representation 1/2. Read it as a coordinate-space operation. Name the input space, the matrix or transformation, and the output space before memorizing formulas. The visible cue is: - Edge-based area representation is possible through implicit edge functions / - Implicit edge function can be defined based on two edge vertices / $\blacksquare = (x, y)$ and $\blacksquare = (x, y)$ / 0 0 0 1 1 1 / - For a given coordinate (x, y) , the implicit edge function is defined as
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Edge-Based Triangle Representation 1/2'?

Page 53	Edge-Based Triangle Representation 2/2
Source cue	- To represent a triangle, three implicit edge functions are combined / - $\blacksquare x, y = y - y \cdot x - x - x \cdot y + \blacksquare \cdot x - x / 0110100110$ / - $\blacksquare x, y = y - y \cdot x - x - x \cdot y + \blacksquare \cdot x - x / 1221211221$
Commentary	This slide is about Edge-Based Triangle Representation 2/2. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - To represent a triangle, three implicit edge functions are combined / - $\blacksquare x, y = y - y \cdot x - x - x \cdot y + \blacksquare \cdot x - x / 0110100110$ / - $\blacksquare x, y = y - y \cdot x - x - x \cdot y + \blacksquare \cdot x - x / 1221211221$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Edge-Based Triangle Representation 2/2'?

Page 54	Rasterization Procedure
Source cue	1. Determine tiles which are (partially) covering triangle / (e.g., easiest approximation would be bounding box) / 2. For each covering tile / - For each pixel / - If all implicit edge functions evaluate to < 0 2

Commentary	This slide is about Rasterization Procedure. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: 1. Determine tiles which are (partially) covering triangle / (e.g., easiest approximation would be bounding box) / 2. For each covering tile / - For each pixel / - If all implicit edge functions evaluate to < 0
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Rasterization Procedure'?

Page 55	between adjacent triangles
Source cue	- Pixel-based displays require the rasterization of primitives / - Rasterization fidelity is critical for image quality / - Rasterization is time-critical and should be optimized for hardware / - Focus on triangles due to speed and ease of tessellation of other polygons / - Segment-based triangle rasterization avoids overlap
Commentary	This slide is about between adjacent triangles. Read it as continuous-to-discrete conversion. The core question is which samples are covered and which interpolated values each fragment receives. The visible cue is: - Pixel-based displays require the rasterization of primitives / - Rasterization fidelity is critical for image quality / - Rasterization is time-critical and should be optimized for hardware / - Focus on triangles due to speed and ease of tessellation of other polygons / - Segment-based triangle rasterization avoids overlap
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'between adjacent triangles'?

Page 56	Literature and other sources used in this chapter
Source cue	The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.
Commentary	This slide is about Literature and other sources used in this chapter. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: The extracted slide text is mostly visual or metadata; use the original PDF page for the diagram or image.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Literature and other sources used in this chapter' into a causal sentence instead of repeating the slide title?

Page 57	Practice, Addison-Wesley.
Source cue	- Text Books / - Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and / Practice, Addison-Wesley. / - Research Publications / - Bresenham, J. E. (1998). Algorithm for computer control of a digital plotter.

Commentary	This slide is about Practice, Addison-Wesley.. Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: - Text Books / - Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and / Practice, Addison-Wesley. / - Research Publications / - Bresenham, J. E. (1998). Algorithm for computer control of a digital plotter.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Practice, Addison-Wesley.' into a causal sentence instead of repeating the slide title?

Page 58	Slide for Future: Sustainability in CG (@ EG25)
Source cue	also @ IEEE VR
Commentary	This slide is about Slide for Future: Sustainability in CG (@ EG25). Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: also @ IEEE VR
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Slide for Future: Sustainability in CG (@ EG25)' into a causal sentence instead of repeating the slide title?

Page 59	Slide for Future: Sustainability in CG (@ EG25)
Source cue	Unoptimized Optimized Unoptimized Optimized / Inverse Simulation of Radiative Thermal Transport / Christian Freude, Lukas Lipp, Matthias Zezulka, Florian Rist, Michael Wimmer, David Hahn
Commentary	This slide is about Slide for Future: Sustainability in CG (@ EG25). Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: Unoptimized Optimized Unoptimized Optimized / Inverse Simulation of Radiative Thermal Transport / Christian Freude, Lukas Lipp, Matthias Zezulka, Florian Rist, Michael Wimmer, David Hahn
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Slide for Future: Sustainability in CG (@ EG25)' into a causal sentence instead of repeating the slide title?

Page 60	Slide for Future: Sustainability in CG (@ EG25)
Source cue	Optimizing Free-Form Grid Shells with Reclaimed Elements under Inventory Constraints / Andrea Favilli, Francesco Laccone, Paolo Cignoni, Luigi Malomo, Daniela Giorgi

Commentary	This slide is about Slide for Future: Sustainability in CG (@ EG25). Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: Optimizing Free-Form Grid Shells with Reclaimed Elements under Inventory Constraints / Andrea Favilli, Francesco Laccone, Paolo Cignoni, Luigi Malomo, Daniela Giorgi
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Slide for Future: Sustainability in CG (@ EG25)' into a causal sentence instead of repeating the slide title?
Page 61	Slide for Future: Sustainability in CG (@ EG25)
Source cue	Stress-Aligned Hexahedral Lattice Structures / Dennis Bukenberger, Junpeng Wang, Jun Wu, Rüdiger Westermann
Commentary	This slide is about Slide for Future: Sustainability in CG (@ EG25). Connect the bullet terms causally: what problem is being solved, what data is used, what output is produced, and which later pipeline stage depends on it. The visible cue is: Stress-Aligned Hexahedral Lattice Structures / Dennis Bukenberger, Junpeng Wang, Jun Wu, Rüdiger Westermann
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Slide for Future: Sustainability in CG (@ EG25)' into a causal sentence instead of repeating the slide title?

06.1 Line Rasterization

Source location: Section 6.1

Commentary

A mathematical line has infinitely many points, but a raster display has a finite grid. Line rasterization decides which grid cells or samples best approximate the ideal line. The algorithm must balance accuracy, speed, and consistency.

Incremental algorithms avoid recomputing expensive formulas for every pixel. The important idea is to step along one axis and update an error term that decides when to step along the other axis.

Mental model

Rasterizing a line means approximating a continuous path by grid decisions.

Check yourself

Why are incremental error updates useful in line rasterization?

06.2 Triangle Edge Rasterization

Source location: Section 6.2

Commentary

Triangles are the core primitive in real-time rendering. Edge functions or half-space tests decide whether a sample lies inside the triangle. This allows the rasterizer to test coverage systematically.

Coverage is not final visibility. A covered sample becomes a fragment candidate. Depth testing, stencil testing, blending, and other operations still decide whether the framebuffer changes.

Mental model

Triangle rasterization answers the question: which samples are inside this projected triangle?

Check yourself

What later stage can reject a fragment after triangle coverage succeeds?

06.3 Region Filling

Source location: Section 6.3

Commentary

Region filling generalizes the idea of determining interior samples. The algorithm must identify connected areas or spans that should receive color. This connects rasterization to scan conversion and image-space reasoning.

The practical issue is avoiding gaps, double fills, or inconsistent boundary handling. Small choices at edges can become visible artifacts when many primitives meet.

Mental model

Filling is about turning boundary descriptions into consistent interior samples.

Check yourself

Why can inconsistent edge rules create cracks between adjacent primitives?

06.4 Scanline-Based Triangle Rasterization

Source location: Section 6.4

Commentary

Scanline rasterization processes horizontal rows of the image. For each row that crosses a triangle, the algorithm finds the covered interval and fills the span. Attribute interpolation can be updated along edges and across each span.

The method is intuitive because it follows the memory layout of many images. It also demonstrates why interpolation is tied to rasterization: once you know a sample location inside the primitive, you can interpolate depth, color, normals, or texture coordinates.

Mental model

Each scanline asks: where does this triangle enter and leave this row?

Check yourself

Which attributes might be interpolated while filling triangle spans?

06.5 Tile-Based Triangle Rasterization

Source location: Section 6.5

Commentary

Tile- or block-based approaches group pixels into small regions. This can improve locality and parallel work distribution. Modern GPUs often reason in blocks or tiles internally because rendering is massively parallel.

For the exam, connect this to efficiency. The mathematical goal is still coverage, but the implementation is organized to use hardware resources well.

Mental model

Block-based rasterization keeps the same coverage problem but changes the work organization.

Check yourself

Why is block organization attractive for parallel graphics hardware?

End-of-lecture summary

If you remember only one thing from Lecture 06, remember this: This lecture explains how continuous primitives become discrete fragments on a grid. Rasterization is the bridge from geometric descriptions to sample-based image generation.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.